Метод ломаных

Поиск глобального минимума методом Пиявского

Трикула Артем Константинович
Поток: МетОпт 1.3

Постановка задачи

Разработать программу поиска глобального минимума одномерной липшицевой функции на заданном отрезке. Входные данные: строка функции, границы отрезка и точность. Результаты: приближённый минимум, аргумент, число итераций, время и визуализация.

$$f(x) = x^2 + 10(1 - \cos(2\pi x)) \to \min, \quad x \in [-2; 2]$$

Использован срез двумерной функции Растригина при y = 0. На выбранном отрезке имеется несколько локальных минимумов, поэтому пример демонстрирует глобальный поиск.

Параметры расчёта

Параметр	Значение
Отрезок	[−2; 2]
Точность по значению функции	ε = 0.01
Интервалы сетки для оценки L	10 000
Коэффициент запаса	K = 1.2
Лимит новых точек	10 000
Настройки	lab02/.env

Контрольный ответ

$$f(x) \geq 0, \quad f(0) = 0 \quad \Rightarrow \quad x^* = 0, \quad f^* = 0.$$

Оба слагаемых неотрицательны. Первое обращается в ноль только при x = 0, поэтому глобальный минимум единственный.

Построение миноранты

Условие Липшица

$$|f(u) - f(v)| \le L|u - v|, \quad u, v \in [a; b].$$

Для каждой вычисленной точки u_i строится вспомогательная функция с двумя прямолинейными ветвями наклонов +L и −L:

$$g_i(x) = f(u_i) - L|x - u_i|.$$

При корректной константе L каждая такая функция лежит не выше исходной. Их максимум образует ломаную миноранту:

$$p_n(x) = \max \{g_0(x), \ldots, g_n(x)\} \le f(x).$$

Начальные точки и выбор следующей

Сначала вычисляются значения в $u_0 = a$ и $u_1 = b$. Верхние вершины ломаной лежат на графике функции. Нижние вершины являются точками пересечения ветвей от соседних верхних вершин.

На каждом шаге выбирается самая низкая нижняя вершина. При равных значениях выбирается самая левая. В её координате вычисляется исходная функция, затем добавляется новая верхняя вершина.

Точное вычисление пересечения

Для двух соседних верхних вершин (x_l, f_l) и (x_r, f_r):

$$f_l - L(t - x_l) = f_r + L(t - x_r),$$

$$t = (x_l + x_r)/2 + (f_l - f_r)/(2L),$$

$$p(t) = (f_l + f_r - L(x_r - x_l))/2.$$

Поиск минимума ломаной выполняется по этим вершинам. Перебор значений на мелкой сетке не заменяет вычисление пересечений.

Оценка L и условие остановки

Однократная оценка перед поиском

$$L = 1.2 \cdot \max_i |(f(x_{i+1}) - f(x_i))/(x_{i+1} - x_i)|.$$

На отрезке [−2; 2] строится равномерная сетка из 10 001 точки. Получено L = 78.3991155461. После этого значение L остаётся постоянным.

Внутренние точки этой сетки не используются как найденные минимумы или верхние вершины. Поиск начинается с концов отрезка; сетка служит только начальной оценке L. Для рисунка применяется отдельная сетка.

Проверка точности

$$\delta_n = f(u_n) - p_{n-1}(u_n) < \varepsilon.$$

Сравнивается значение функции со значением предыдущей ломаной в той же точке. После добавления новой вершины ломаная проходит через неё, и проверяемая разность стала бы нулевой.

$$p_{n-1}(u_n) \le f^* \le f(u_n).$$

Если L удовлетворяет условию Липшица, разность δ_n ограничивает ошибку по значению функции. Она не задаёт непосредственную погрешность координаты x.

Обоснование для выбранного примера

$$f'(x) = 2x + 20\pi \sin(2\pi x),$$

$$|f'(x)| \le 4 + 20\pi \approx 66.8318531 < L \approx 78.3991155.$$

Для демонстрационной функции полученное L превышает аналитическую верхнюю границу производной. Следовательно, миноранта корректна и оценка точности обоснована.

Для произвольной функции конечная сетка с запасом 20% такой гарантии не даёт. При обнаружении большего наклона в новой точке программа завершает поиск с сообщением invalid-lipschitz, не меняя L.

Программная реализация

Задача решена на Python собственным методом Пиявского.

Порядок работы программы

1. Чтение строковых настроек из lab02/.env и проверка параметров.
2. Разбор математического выражения с разрешёнными операциями.
3. Однократная оценка константы L с фиксированным K = 1.2.
4. Расчёт пересечений, выбор нижней вершины и уточнение ломаной.
5. Проверка разрыва, сохранение истории и лучшей вычисленной точки.
6. Вывод таблицы, численных результатов и сохранение графика.

Структура исходного кода

Файл	Назначение
models.py	Задача, точки, итерации и результат
config.py	Чтение .env и проверка параметров
expression.py	Разбор, проверка и вычисление выражения
piyavsky.py	Оценка L и поиск по вершинам ломаной
console_tables.py	Таблица шагов и итоговые показатели
plotting.py	Графики функции, миноранты и разрыва
main.py	Последовательный сценарий запуска

Настройки и запуск

Данные читаются из .env без интерактивного ввода. В репозитории хранится .env.example; локальный .env исключён из Git. Команды выполняются из корня проекта.

```
poetry install
poetry run python lab02/src/main.py
```

График сохраняется в lab02/imgs/solution.png. Время расчёта измеряется без вывода таблиц и построения графика. Граничные значения переиспользуются из этапа оценки L.

Результаты расчёта

Показатель	Результат
Статус	converged
Лучшая точка x	0
Значение f(x)	0
Число итераций	180
Время расчётов	0.032837 с

Время относится к одному запуску и зависит от компьютера и нагрузки. Время вывода таблицы и построения графика не учитывается.

Последняя проверка точности

$u_{180} = -0.004380226746$

$f(u_{180}) = 0.003806188202$

p предыдущей ломаной = -0.005969235320

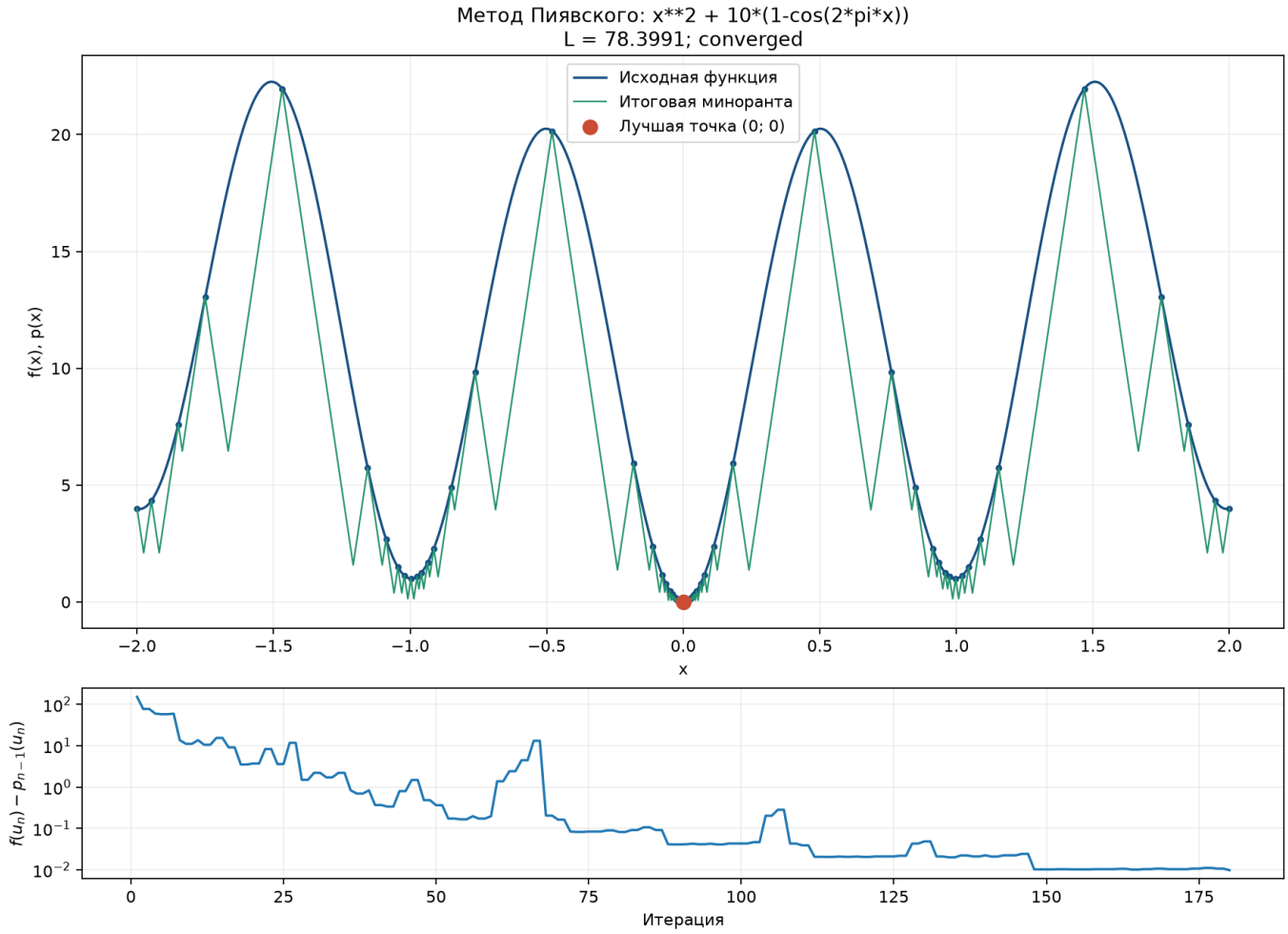
$\delta = 0.009775423521 < 0.01$

Последняя проверочная точка отличается от лучшей найденной. Программа возвращает лучшую точку всей истории, включая границы. Точка x = 0 найдена раньше, но поиск продолжался до подтверждения точности минорантой.

Вывод

Получен известный глобальный минимум функции Растригина: x* = 0, f* = 0. Итерационный процесс выполнен по аналитически вычисленным пересечениям ветвей. Критерий остановки выполнен при фиксированном L.

Визуализация результата



Верхний график показывает исходную функцию, итоговую миноранту и найденную точку минимума. Сгущение верхних вершин происходит в областях, где нижняя оценка допускает более низкое значение функции.

Нижний график показывает разрыв между исходной функцией и предыдущей минорантой в очередной точке поиска. Разрыв не обязан убывать на каждом шаге: алгоритм переходит между разными интервалами.

Ломаная изображена по вычисленным верхним и нижним вершинам. Точки равномерной сетки используются только для отображения гладкого графика функции.

История итераций / 1-30

Шаг	u_n	$f(u_n)$	$p_{n-1}(u_n)$	Разрыв
1	0.00000000	0.00000000	-152.79823109	152.79823109
2	-0.97448951	1.07781485	-76.39911555	77.47693040
3	0.97448951	1.07781485	-76.39911555	77.47693040
4	-1.46860816	21.96292016	-37.66065035	59.62357051
5	-0.48037086	20.15479671	-37.66065035	57.81544705
6	0.48037086	20.15479671	-37.66065035	57.81544705
7	1.46860816	21.96292016	-37.66065035	59.62357051
8	-0.84909598	4.88915919	-8.75292682	13.64208601
9	-0.11164573	2.37365563	-8.75292682	11.12658245
10	0.11164573	2.37365563	-8.75292682	11.12658245
11	0.84909598	4.88915919	-8.75292682	13.64208601
12	-1.08835151	2.68618382	-7.84886509	10.53504892
13	1.08835151	2.68618382	-7.84886509	10.53504892
14	-1.84886481	7.59830153	-7.84886509	15.44716662
15	1.84886481	7.59830153	-7.84886509	15.44716662
16	-0.18260688	5.92432323	-3.18963560	9.11395883
17	0.18260688	5.92432323	-3.18963560	9.11395883
18	-0.04068459	0.32661030	-3.18963560	3.51624590
19	0.04068459	0.32661030	-3.18963560	3.51624590
20	-1.02116294	1.13104953	-2.58134063	3.71239016
21	1.02116294	1.13104953	-2.58134063	3.71239016
22	-1.15554008	5.74253754	-2.58134063	8.32387817
23	1.15554008	5.74253754	-2.58134063	8.32387817
24	-0.93610006	1.67150609	-1.93188381	3.60338990
25	0.93610006	1.67150609	-1.93188381	3.60338990
26	-0.76209190	9.82175834	-1.93188381	11.75364215
27	0.76209190	9.82175834	-1.93188381	11.75364215
28	-0.01825930	0.06607214	-1.43151265	1.49758479
29	0.01825930	0.06607214	-1.43151265	1.49758479
30	-0.06310988	0.77991964	-1.43151265	2.21143229

Стартовые точки a = −2 и b = 2 не включены в нумерацию новых точек.

История итераций / 31-60

Шаг	u_n	$f(u_n)$	$p_{n-1}(u_n)$	Разрыв
31	0.06310988	0.77991964	-1.43151265	2.21143229
32	-0.99748671	0.99622656	-0.72514555	1.72137212
33	0.99748671	0.99622656	-0.72514555	1.72137212
34	-1.04483916	1.48593749	-0.72514555	2.21108304
35	1.04483916	1.48593749	-0.72514555	2.21108304
36	-0.02781033	0.15305122	-0.68272025	0.83577148
37	-0.00870826	0.01504111	-0.68272025	0.69776136
38	0.00870826	0.01504111	-0.68272025	0.69776136
39	0.02781033	0.15305122	-0.68272025	0.83577148
40	-0.01315832	0.03433044	-0.33383957	0.36817001
41	0.01315832	0.03433044	-0.33383957	0.36817001
42	-0.00425821	0.00359709	-0.33383957	0.33743667
43	0.00425821	0.00359709	-0.33383957	0.33743667
44	-0.04900620	0.47272617	-0.32579650	0.79852267
45	0.04900620	0.47272617	-0.32579650	0.79852267
46	-0.07721356	1.15989831	-0.32579650	1.48569481
47	0.07721356	1.15989831	-0.32579650	1.48569481
48	-0.03314056	0.21711121	-0.26483451	0.48194572
49	0.03314056	0.21711121	-0.26483451	0.48194572
50	-0.02248009	0.10009260	-0.26483451	0.36492712
51	0.02248009	0.10009260	-0.26483451	0.36492712
52	-0.00641025	0.00815109	-0.16512124	0.17327233
53	0.00641025	0.00815109	-0.16512124	0.17327233
54	-0.00210616	0.00088004	-0.16512124	0.16600128
55	0.00210616	0.00088004	-0.16512124	0.16600128
56	-0.01550637	0.04766537	-0.14975457	0.19741994
57	-0.01081027	0.02317563	-0.14975457	0.17293020
58	0.01081027	0.02317563	-0.14975457	0.17293020
59	0.01550637	0.04766537	-0.14975457	0.19741994
60	-0.95908112	1.24852438	-0.13018886	1.37871324

Стартовые точки a = −2 и b = 2 не включены в нумерацию новых точек.

История итераций / 61-90

Шаг	u_n	$f(u_n)$	$p_{n-1}(u_n)$	Разрыв
61	0.95908112	1.24852438	-0.13018886	1.37871324
62	-0.91311900	2.28712773	-0.13018886	2.41731660
63	0.91311900	2.28712773	-0.13018886	2.41731660
64	-1.94738102	4.33386347	-0.12528178	4.45914526
65	1.94738102	4.33386347	-0.12528178	4.45914526
66	-1.75034861	13.04181664	-0.12528178	13.16709842
67	1.75034861	13.04181664	-0.12528178	13.16709842
68	-0.02480746	0.12184674	-0.08237095	0.20421769
69	0.02480746	0.12184674	-0.08237095	0.20421769
70	-0.02015272	0.08046638	-0.08237095	0.16283733
71	0.02015272	0.08046638	-0.08237095	0.16283733
72	-0.00316486	0.00198709	-0.08212060	0.08410769
73	-0.00104747	0.00021767	-0.08212060	0.08233827
74	0.00104747	0.00021767	-0.08212060	0.08233827
75	0.00316486	0.00198709	-0.08212060	0.08410769
76	-0.00530518	0.00558323	-0.07848507	0.08406830
77	0.00530518	0.00558323	-0.07848507	0.08406830
78	-0.00751532	0.01120311	-0.07848507	0.08968818
79	0.00751532	0.01120311	-0.07848507	0.08968818
80	-0.00970739	0.01868940	-0.06328947	0.08197887
81	0.00970739	0.01868940	-0.06328947	0.08197887
82	-0.01191316	0.02814339	-0.06328947	0.09143285
83	0.01191316	0.02814339	-0.06328947	0.09143285
84	-0.01676544	0.05571278	-0.05104460	0.10675738
85	0.01676544	0.05571278	-0.05104460	0.10675738
86	-0.01424730	0.04024400	-0.05104460	0.09128860
87	0.01424730	0.04024400	-0.05104460	0.09128860
88	-0.00157259	0.00049063	-0.04095146	0.04144209
89	-0.00052235	0.00005413	-0.04095146	0.04100559
90	0.00052235	0.00005413	-0.04095146	0.04100559

Стартовые точки a = −2 и b = 2 не включены в нумерацию новых точек.

История итераций / 91-120

Шаг	u_n	$f(u_n)$	$p_{n-1}(u_n)$	Разрыв
91	0.00157259	0.00049063	-0.04095146	0.04144209
92	-0.00370126	0.00271772	-0.04006675	0.04278447
93	-0.00262845	0.00137061	-0.04006675	0.04143736
94	0.00262845	0.00137061	-0.04006675	0.04143736
95	0.00370126	0.00271772	-0.04006675	0.04278447
96	-0.00476903	0.00451182	-0.03645092	0.04096274
97	0.00476903	0.00451182	-0.03645092	0.04096274
98	-0.00584134	0.00676863	-0.03645092	0.04321955
99	0.00584134	0.00676863	-0.03645092	0.04321955
100	-0.00694332	0.00956291	-0.03364098	0.04320389
101	0.00694332	0.00956291	-0.03364098	0.04320389
102	-0.00808731	0.01297298	-0.03364098	0.04661397
103	0.00808731	0.01297298	-0.03364098	0.04661397
104	-0.03006689	0.17881996	-0.02386165	0.20268161
105	0.03006689	0.17881996	-0.02386165	0.20268161
106	-0.03621423	0.25907037	-0.02386165	0.28293202
107	0.03621423	0.25907037	-0.02386165	0.28293202
108	-0.01023022	0.02075609	-0.02230003	0.04305612
109	0.01023022	0.02075609	-0.02230003	0.04305612
110	-0.00918456	0.01673097	-0.02230003	0.03903100
111	0.00918456	0.01673097	-0.02230003	0.03903100
112	-0.00078386	0.00012190	-0.02044867	0.02057057
113	0.00078386	0.00012190	-0.02044867	0.02057057
114	-0.00026083	0.00001350	-0.02044867	0.02046216
115	0.00026083	0.00001350	-0.02044867	0.02046216
116	-0.00183689	0.00066940	-0.02023042	0.02089982
117	-0.00130829	0.00033957	-0.02023042	0.02056999
118	0.00130829	0.00033957	-0.02023042	0.02056999
119	0.00183689	0.00066940	-0.02023042	0.02089982
120	-0.00236418	0.00110886	-0.01934807	0.02045693

Стартовые точки $a = -2$ и $b = 2$ не включены в нумерацию новых точек.

История итераций / 121-150

Шаг	u_n	$f(u_n)$	$p_{n-1}(u_n)$	Разрыв
121	0.00236418	0.00110886	-0.01934807	0.02045693
122	-0.00289272	0.00166007	-0.01934807	0.02100814
123	0.00289272	0.00166007	-0.01934807	0.02100814
124	-0.00342840	0.00233180	-0.01867452	0.02100631
125	0.00342840	0.00233180	-0.01867452	0.02100631
126	-0.00397413	0.00313318	-0.01867452	0.02180770
127	0.00397413	0.00313318	-0.01867452	0.02180770
128	-0.01133003	0.02545683	-0.01757304	0.04302986
129	0.01133003	0.02545683	-0.01757304	0.04302986
130	-0.01249628	0.03096449	-0.01757304	0.04853753
131	0.01249628	0.03096449	-0.01757304	0.04853753
132	-0.00503027	0.00501963	-0.01596955	0.02098918
133	0.00503027	0.00501963	-0.01596955	0.02098918
134	-0.00450778	0.00403108	-0.01596955	0.02000063
135	0.00450778	0.00403108	-0.01596955	0.02000063
136	-0.00611698	0.00742241	-0.01484115	0.02226356
137	0.00611698	0.00742241	-0.01484115	0.02226356
138	-0.00556570	0.00614498	-0.01484115	0.02098612
139	0.00556570	0.00614498	-0.01484115	0.02098612
140	-0.00721886	0.01033682	-0.01203904	0.02237586
141	-0.00666778	0.00881909	-0.01203904	0.02085813
142	0.00666778	0.00881909	-0.01203904	0.02085813
143	0.00721886	0.01033682	-0.01203904	0.02237586
144	-0.00779003	0.01203694	-0.01033400	0.02237094
145	0.00779003	0.01203694	-0.01033400	0.02237094
146	-0.00838460	0.01394405	-0.01033400	0.02427805
147	0.00838460	0.01394405	-0.01033400	0.02427805
148	-0.00039133	0.00003038	-0.01021758	0.01024797
149	0.00039133	0.00003038	-0.01021758	0.01024797
150	-0.00013033	0.00000337	-0.01021758	0.01022095

Стартовые точки a = −2 и b = 2 не включены в нумерацию новых точек.

История итераций / 151-180

Шаг	u_n	$f(u_n)$	$p_{n-1}(u_n)$	Разрыв
151	0.00013033	0.00000337	-0.01021758	0.01022095
152	-0.00091506	0.00016612	-0.01016338	0.01032950
153	0.00091506	0.00016612	-0.01016338	0.01032950
154	-0.00065267	0.00008451	-0.01016338	0.01024789
155	0.00065267	0.00008451	-0.01016338	0.01024789
156	-0.00117710	0.00027488	-0.00994542	0.01022031
157	0.00117710	0.00027488	-0.00994542	0.01022031
158	-0.00143948	0.00041108	-0.00994542	0.01035651
159	0.00143948	0.00041108	-0.00994542	0.01035651
160	-0.00170360	0.00057578	-0.00978051	0.01035629
161	0.00170360	0.00057578	-0.00978051	0.01035629
162	-0.00197018	0.00077007	-0.00978051	0.01055058
163	0.00197018	0.00077007	-0.00978051	0.01055058
164	-0.00223371	0.00098985	-0.00911961	0.01010946
165	0.00223371	0.00098985	-0.00911961	0.01010946
166	-0.00249464	0.00123462	-0.00911961	0.01035422
167	0.00249464	0.00123462	-0.00911961	0.01035422
168	-0.00302670	0.00181740	-0.00884400	0.01066140
169	0.00302670	0.00181740	-0.00884400	0.01066140
170	-0.00275874	0.00150985	-0.00884400	0.01035386
171	0.00275874	0.00150985	-0.00884400	0.01035386
172	-0.00329443	0.00215313	-0.00817136	0.01032449
173	0.00329443	0.00215313	-0.00817136	0.01032449
174	-0.00356237	0.00251759	-0.00817136	0.01068895
175	0.00356237	0.00251759	-0.00817136	0.01068895
176	-0.00411321	0.00335631	-0.00777067	0.01112697
177	0.00411321	0.00335631	-0.00777067	0.01112697
178	-0.00383504	0.00291772	-0.00777067	0.01068839
179	0.00383504	0.00291772	-0.00777067	0.01068839
180	-0.00438023	0.00380619	-0.00596924	0.00977542

Стартовые точки $a = -2$ и $b = 2$ не включены в нумерацию новых точек.

Листинг: .env.example

```
# Минимум f(0)=0.
FUNCTION="x**2 + 10*(1-cos(2*pi*x))"
A=-2
B=2
EPS=0.01
ESTIMATION_INTERVALS=10000
MAX_ITERATIONS=10000
```

Листинг: main.py

```python
from pathlib import Path
import sys

from rich.console import Console
from rich.text import Text

from models import OptimizationProblem
from config import Settings
from console_tables import print_solution
from expression import Expression
from piyavsky import PiyavskySolver
from plotting import save_plot


def main() -> int:
    """Запускает расчёт, вывод таблицы и сохранение графика

    :return: 0 при достижении точности, 1 при ошибке или досрочной остановке
    """
    directory = Path(__file__).resolve().parents[1]
    # UTF-8 сохраняет русские подписи и математические индексы при перенаправлении.
    if hasattr(sys.stdout, "reconfigure"):
        sys.stdout.reconfigure(encoding="utf-8")
    try:
        settings = Settings.read(directory / ".env")
        function = Expression(settings.expression)
        problem = OptimizationProblem(function=function, a=settings.a, b=settings.b)
        solver = PiyavskySolver(
            tolerance=settings.eps,
            estimation_intervals=settings.estimation_intervals,
            max_iterations=settings.max_iterations,
        )
        result = solver.solve(problem)

        print_solution(result)
        destination = directory / "imgs" / "solution.png"
        save_plot(function, settings.expression, result, destination)
        Console().print(Text.assemble("График: ", (str(destination), "bright_cyan")))
        return 0 if result.status == "converged" else 1
    except (ValueError, OSError) as error:
        print(f"Ошибка: {error}")
        return 1


if __name__ == "__main__":
    raise SystemExit(main())

```

Листинг: models.py

```python
from collections.abc import Callable
from math import isfinite
from dataclasses import dataclass
from typing import Literal


@dataclass
class OptimizationProblem:
    """Исходная задача минимизации на отрезке

    :param function: Вычисляемая функция одной переменной
    :param a: Левая граница отрезка
    :param b: Правая граница отрезка
    """

    function: Callable[[float], float]
    a: float
    b: float

    def __post_init__(self):
        """Проверяет конечность и порядок границ

        :return: None
        """
        if not all(isfinite(value) for value in (self.a, self.b, self.b - self.a)):
            raise ValueError("Границы и длина отрезка должны быть конечными.")
        if self.a >= self.b:
            raise ValueError("Требуется a < b.")

    def evaluate(self, x: float) -> "Point":
        """Вычисляет вещественное конечное значение функции

        :param x: Аргумент функции
        :return: Точка графика функции
        """
        value = float(self.function(x))
        if not isfinite(value):
            raise ValueError(f"Неконечное значение функции при x={x:g}.")
        return Point(x, value)


@dataclass(frozen=True)
class Point:
    """Точка графика функции или вершина миноранты

    :param x: Координата по оси аргумента
```

Листинг: models.py

```
47      :param y: Значение функции или миноранты
48      """
49
50      x: float
51      y: float
52
53
54  @dataclass(frozen=True)
55  class Iteration:
56      """Снимок одной завершённой итерации
57
58      :param number: Номер новой точки после двух стартовых
59      :param point: Вычисленная точка исходной функции
60      :param lower: Значение предыдущей миноранты в новой точке
61      :param gap: Разность point.y - lower до обновления миноранты
62      """
63
64      number: int
65      point: Point
66      lower: float
67      gap: float
68
69
70  @dataclass
71  class Solution:
72      """Результат поиска с историей и данными для визуализации
73
74      :param status: converged, iteration-limit, invalid-lipschitz или stalled
75      :param point: Лучшая принятая точка, включая границы
76      :param lipschitz: Однократно оценённая константа L
77      :param history: Снимки завершённых итераций
78      :param points: Принятые верхние вершины в порядке возрастания x
79      :param estimation_evaluations: Число вычислений для оценки L, включая границы
80      :param search_evaluations: Дополнительные вычисления при поиске
81      :param elapsed: Время оценки L и поиска в секундах без вывода и графиков
82      """
83
84      status: Literal["converged", "iteration-limit", "invalid-lipschitz", "stalled"]
85      point: Point
86      lipschitz: float
87      history: list[Iteration]
88      points: list[Point]
89      estimation_evaluations: int
90      search_evaluations: int
91      elapsed: float
92
```

Листинг: config.py

```python
from dataclasses import dataclass
from math import isfinite
from pathlib import Path

from dotenv import dotenv_values


@dataclass(frozen=True)
class Settings:
    """Строковые настройки .env после преобразования в числовые типы

    :param expression: Математическое выражение от x
    :param a: Левая граница
    :param b: Правая граница
    :param eps: Точность по значению функции
    :param estimation_intervals: Число интервалов начальной сетки
    :param max_iterations: Лимит новых точек поиска
    """

    expression: str
    a: float
    b: float
    eps: float
    estimation_intervals: int
    max_iterations: int

    @classmethod
    def read(cls, path: Path) -> "Settings":
        """Читает обязательные параметры и проверяет их значения

        :param path: Путь к локальному файлу .env
```

Листинг: config.py

```
        :return: Проверенные настройки запуска
        """
        if not path.is_file():
            raise ValueError(f"Нет {path}. Скопируйте lab02/.env.example в lab02/.env.")
        values = dotenv_values(path, encoding="utf-8-sig", interpolate=False)
        keys = ("FUNCTION", "A", "B", "EPS", "ESTIMATION_INTERVALS", "MAX_ITERATIONS")
        missing = [key for key in keys if not values.get(key)]
        if missing:
            raise ValueError("Не заданы параметры: " + ", ".join(missing))
        try:
            result = cls(
                expression=values["FUNCTION"],
                a=float(values["A"]),
                b=float(values["B"]),
                eps=float(values["EPS"]),
                estimation_intervals=int(values["ESTIMATION_INTERVALS"]),
                max_iterations=int(values["MAX_ITERATIONS"]),
            )
        except (TypeError, ValueError) as error:
            raise ValueError("Проверьте числовые параметры .env; десятичный разделитель — точка.") from error
        if not all(isfinite(x) for x in (result.a, result.b, result.eps, result.b - result.a)):
            raise ValueError("Границы, длина отрезка и EPS должны быть конечными.")
        if result.a >= result.b or result.eps <= 0:
            raise ValueError("Требуется A < B и EPS > 0.")
        if result.estimation_intervals < 2 or result.max_iterations < 1:
            raise ValueError("ESTIMATION_INTERVALS >= 2, MAX_ITERATIONS >= 1.")
        return result

```

Листинг: expression.py

```
import ast
import math


FUNCTIONS = {name: getattr(math, name) for name in (
    "sin", "cos", "tan", "exp", "log", "log10", "sqrt", "sinh", "cosh", "tanh"
)}
FUNCTIONS["abs"] = abs
CONSTANTS = {"pi": math.pi, "e": math.e}
ALLOWED_NAMES = {"x", *FUNCTIONS, *CONSTANTS}
ALLOWED_NODES = (
    ast.Expression,
    ast.BinOp, ast.UnaryOp,
    ast.Add, ast.Sub, ast.Mult, ast.Div, ast.Pow,
    ast.UAdd, ast.USub,
    ast.Name, ast.Constant, ast.Load,
)


class Expression:
    """Математическое выражение с одной переменной и разрешёнными операциями."""

    def __init__(self, source: str):
        """Проверяет синтаксис выражения и компилирует его

        :param source: Выражение с x, разрешёнными функциями и константами
        :return: None
        """
        if len(source) > 1000:
            raise ValueError("Слишком длинное выражение функции.")
        try:
            tree = ast.parse(source, mode="eval")
        except SyntaxError as error:
            raise ValueError("Некорректная запись FUNCTION.") from error
        for node in ast.walk(tree):
            self._validate_node(node)

        self.code = compile(tree, "<function>", "eval")

    def _validate_node(self, node: ast.AST) -> None:
        """Проверяет один элемент синтаксического дерева

        :param node: Операция, имя, число или вызов функции
        :return: None
        """
```

Листинг: expression.py

```
46          if isinstance(node, ast.Call):
47              self._validate_call(node)
48              return
49
50          if isinstance(node, ast.Name) and node.id not in ALLOWED_NAMES:
51              raise ValueError(f"Неизвестное имя: {node.id}")
52
53          if isinstance(node, ast.Constant) and type(node.value) not in (int, float):
54              raise ValueError("В выражении разрешены только числа.")
55
56          if not isinstance(node, ALLOWED_NODES):
57              raise ValueError(f"Недопустимая операция: {type(node).__name__}")
58
59      def _validate_call(self, node: ast.Call) -> None:
60          """Проверяет имя функции и единственный позиционный аргумент
61
62          :param node: Вызов функции в выражении
63          :return: None
64          """
65          if not isinstance(node.func, ast.Name):
66              raise ValueError("Функцию нужно вызывать по имени, например sin(x).")
67
68          if node.func.id not in FUNCTIONS:
69              raise ValueError(f"Недопустимая функция: {node.func.id}")
70
71          if len(node.args) != 1 or node.keywords:
72              raise ValueError("Функция должна иметь один позиционный аргумент.")
73
74      def __call__(self, x: float) -> float:
75          """Вычисляет выражение в заданной точке
76
77          :param x: Аргумент функции
78          :return: Конечное вещественное значение выражения
79          """
80          variables = {**FUNCTIONS, **CONSTANTS, "x": x}
81          try:
82              value = float(eval(self.code, {"__builtins__": {}}, variables))
83          except (ArithmeticError, ValueError, TypeError) as error:
84              raise ValueError(f"Функция не определена как вещественное число при x={x:g}.")
    from error
85          if not math.isfinite(value):
86              raise ValueError(f"Получено бесконечное значение функции при x={x:g}.")
87          return value
88
```

Листинг: piyavsky.py

```python
from math import isfinite
from time import perf_counter

from models import Iteration, OptimizationProblem, Point, Solution


def intersection(left: Point, right: Point, lipschitz: float) -> Point:
    """Вычисляет нижнюю вершину между соседними верхними вершинами

    :param left: Верхняя вершина слева
    :param right: Верхняя вершина справа
    :param lipschitz: Положительная константа наклона L
    :return: Точка пересечения ветвей миноранты
    """
    width = right.x - left.x
    midpoint = left.x + width / 2
    correction = (left.y - right.y) / (2 * lipschitz)
    lower_value = left.y / 2 + right.y / 2 - lipschitz * width / 2
    return Point(midpoint + correction, lower_value)


class PiyavskySolver:
    """Метод ломаных с фиксированной оценкой L и историей итераций"""

    def __init__(self, tolerance=0.01, estimation_intervals=10000, max_iterations=10000):
        """Настраивает точность и ограничения вычислений

        :param tolerance: Положительная точность по значению функции
        :param estimation_intervals: Число интервалов сетки для оценки L
        :param max_iterations: Максимальное число новых точек поиска
        :return: None
        """
        if not isfinite(tolerance) or tolerance <= 0:
            raise ValueError("Точность должна быть положительной и конечной.")
        if type(estimation_intervals) is not int or estimation_intervals < 2:
            raise ValueError("Число интервалов должно быть целым и не меньше 2.")
```

Листинг: piyavsky.py

```
37          if type(max_iterations) is not int or max_iterations < 1:
38              raise ValueError("Лимит итераций должен быть положительным целым числом.")
39          self.tolerance = tolerance
40          self.estimation_intervals = estimation_intervals
41          self.max_iterations = max_iterations
42
43      def _estimate_lipschitz(self, problem: OptimizationProblem):
44          """Один раз оценивает L по сетке с коэффициентом запаса 1.2
45
46          :param problem: Исходная задача минимизации
47          :return: Оценка L и две стартовые точки на границах отрезка
48          """
49          first = problem.evaluate(problem.a)
50          previous = first
51          width = problem.b - problem.a
52          largest_slope = 0.0
53
54          for i in range(1, self.estimation_intervals + 1):
55              x = problem.a + width * i / self.estimation_intervals
56              if i == self.estimation_intervals:
57                  x = problem.b
58              current = problem.evaluate(x)
59              delta_x = current.x - previous.x
60              delta_y = current.y - previous.y
61
62              slope = abs(delta_y / delta_x)
63              largest_slope = max(largest_slope, slope)
64              previous = current
65
66          lipschitz = 1.2 * largest_slope
67
68          if lipschitz == 0:
69              lipschitz = min(1.0, self.tolerance) / max(1.0, width)
70          if not isfinite(lipschitz) or lipschitz <= 0:
71              raise ValueError("Не удалось получить конечную положительную оценку L.")
72          return lipschitz, [first, previous]
```

Листинг: piyavsky.py

```
73
74      def _optimize(self, problem: OptimizationProblem, lipschitz: float, points: list[Point]):
75          """Уточняет миноранту до выполнения критерия точности или остановки
76
77          :param problem: Исходная задача
78          :param lipschitz: Фиксированная оценка L
79          :param points: Верхние вершины; список обновляется при поиске
80          :return: Статус, лучшая точка, история и число новых вычислений
81          """
82          history = []
83          best = min(points, key=lambda point: point.y)
84          evaluations = 0
85          status = "iteration-limit"
86
87          for number in range(1, self.max_iterations + 1):
88              candidates = [
89                  intersection(left, right, lipschitz)
90                  for left, right in zip(points, points[1:])
91              ]
92              index = min(
93                  range(len(candidates)),
94                  key=lambda i: (candidates[i].y, candidates[i].x),
95              )
96              lower = candidates[index]
97              left, right = points[index:index + 2]
98              if not left.x < lower.x < right.x:
99                  status = "stalled"
100                 break
101
102             point = problem.evaluate(lower.x)
103             evaluations += 1
104             left_slope = abs((point.y - left.y) / (point.x - left.x))
105             right_slope = abs((right.y - point.y) / (right.x - point.x))
106
107             if max(left_slope, right_slope) > lipschitz * (1 + 1e-10):
108                 status = "invalid-lipschitz"
```

Листинг: piyavsky.py

```
                break

            gap = point.y - lower.y
            history.append(Iteration(number, point, lower.y, gap))
            points.insert(index + 1, point)
            if point.y < best.y:
                best = point

            if gap < self.tolerance:
                status = "converged"
                break

        return status, best, history, evaluations

    def solve(self, problem: OptimizationProblem) -> Solution:
        """Оценивает L, запускает поиск и собирает результат

        :param problem: Функция и отрезок минимизации
        :return: Ответ, история, счётчики вычислений и время расчёта
        """
        started = perf_counter()
        lipschitz, points = self._estimate_lipschitz(problem)
        status, best, history, evaluations = self._optimize(problem, lipschitz, points)
        elapsed = perf_counter() - started

        return Solution(
            status=status,
            point=best,
            lipschitz=lipschitz,
            history=history,
            points=points,
            estimation_evaluations=self.estimation_intervals + 1,
            search_evaluations=evaluations,
            elapsed=elapsed,
        )
```

Листинг: console_tables.py

```python
from rich import box
from rich.console import Console
from rich.table import Table
from rich.text import Text

from models import Solution


def print_solution(result: Solution) -> None:
    """Показывает историю поиска и итоговые показатели

    :param result: Результат решения с историей итераций
    :return: None
    """
    console = Console()
    table = Table(
        title="Метод Пиявского",
        box=box.ROUNDED,
        header_style="bold cyan",
        border_style="dim",
    )
    for title in ("Шаг", "uₙ", "f(uₙ)", "pₙ₋₁(uₙ)", "Разрыв"):
        table.add_column(title, justify="right")

    for step in result.history:
        values = (step.point.x, step.point.y, step.lower, step.gap)
        table.add_row(str(step.number), *(f"{value:.9g}" for value in values))

    console.print(table)
    console.print(Text.assemble("Статус: ", (result.status, "bright_cyan")))
    console.print(Text.assemble(
        "Найденный минимум: ",
        (f"x = {result.point.x:.12g}, ", "bright_cyan"),
        ("f", "magenta"),
        (f"(x) = {result.point.y:.12g}", "bright_cyan"),
    ))
    console.print(Text.assemble("Число итераций: ", (str(len(result.history)),
"bright_cyan")))
    console.print(Text.assemble("Время расчётов: ", (f"{result.elapsed:.6f} с",
"bright_cyan")))


```

Листинг: plotting.py

```
1  from collections.abc import Callable
2  from pathlib import Path
3
4  import matplotlib
5
6  matplotlib.use("Agg")
7  import matplotlib.pyplot as plt
8  from matplotlib.axes import Axes
9  import numpy as np
10
11 from models import Solution
12 from piyavsky import intersection
13
14
15 def _plot_function(axes: Axes, function: Callable[[float], float],
16                    expression: str, result: Solution) -> None:
17     """Рисует функцию, итоговую миноранту и найденный минимум
18
19     :param axes: Область графика функции
20     :param function: Вычисляемая исходная функция
21     :param expression: Текст функции для заголовка
22     :param result: Результат поиска с упорядоченными точками
23     :return: None
24     """
25     points = result.points
26     arguments = np.linspace(points[0].x, points[-1].x, 2001)
27     values = [function(float(argument)) for argument in arguments]
28
29     # Ломаная чередует принятые точки и пересечения соседних ветвей.
30     vertices = [points[0]]
31     for left, right in zip(points, points[1:]):
32         lower_vertex = intersection(left, right, result.lipschitz)
33         vertices.extend([lower_vertex, right])
34
35     minorant_x = [vertex.x for vertex in vertices]
36     minorant_y = [vertex.y for vertex in vertices]
37     sampled_x = [point.x for point in points]
38     sampled_y = [point.y for point in points]
39     best = result.point
40     best_label = f"Лучшая точка ({best.x:.6g}; {best.y:.6g})"
41
42     axes.plot(arguments, values, label="Исходная функция", color="#164e83")
43     axes.plot(minorant_x, minorant_y, label="Итоговая миноранта", color="#24936e",
   linewidth=1)
44     axes.scatter(sampled_x, sampled_y, s=9, color="#164e83")
45     axes.scatter(best.x, best.y, s=75, color="#cb4b33", zorder=5, label=best_label)
46
```

Листинг: plotting.py

```
47      title = f"Метод Пиявского: {expression}\nL = {result.lipschitz:.6g}; {result.status}"
48      axes.set(title=title, xlabel="x", ylabel="f(x), p(x)")
49      axes.legend()
50      axes.grid(alpha=0.2)
51
52
53  def _plot_convergence(axes: Axes, result: Solution) -> None:
54      """Рисует разрыв между функцией и минорантой по итерациям
55
56      :param axes: Область графика сходимости
57      :param result: Результат поиска с историей итераций
58      :return: None
59      """
60      iterations = [step.number for step in result.history]
61      gaps = [step.gap for step in result.history]
62      axes.plot(iterations, gaps)
63
64      if result.history:
65          axes.set_yscale("symlog", linthresh=1e-10)
66
67      axes.set(xlabel="Итерация", ylabel=r"$f(u_n) - p_{n-1}(u_n)$")
68      axes.grid(alpha=0.2)
69
70
71  def save_plot(function: Callable[[float], float], expression: str,
72                result: Solution, destination: Path) -> None:
73      """Сохраняет функцию, миноранту и историю разрыва на одном рисунке
74
75      :param function: Вычисляемая исходная функция
76      :param expression: Текст функции для заголовка
77      :param result: Результат поиска и принятые вершины
78      :param destination: Путь сохранения PNG
79      :return: None
80      """
81      figure, (function_axes, convergence_axes) = plt.subplots(
82          2, 1,
83          figsize=(11, 8),
84          layout="constrained",
85          gridspec_kw={"height_ratios": [3, 1]},
86      )
87      try:
88          _plot_function(function_axes, function, expression, result)
89          _plot_convergence(convergence_axes, result)
90          destination.parent.mkdir(parents=True, exist_ok=True)
91          figure.savefig(destination, dpi=160)
92      finally:
93          plt.close(figure)
```